COLLEGE OF TECHNOLOGY AND BUILT ENVIRONMENT

SCHOOL OF INFORMATION TECHNOLOGY AND ENGINEERING

DEPARTMENT OF IT/SE ENG.

ModIntel - AI Enhanced ModSecurity

Software Design Specification

Team Members

Meaza Tadele	UGR/6378/14
Fozia Mohammed	UGR/4535/14
Liyu Desta	UGR/6694/13
Natanim Kemal	UGR/4648/14
Eyoab Amare	UGR/5756/14

Advisors: Mr. Nebiat

Contents

List of Tables 3
List of figures 4
Definitions, Acronyms, Abbreviations 5
1. Introduction 6
1.1 Purpose 6
1.2 General Overview 6
1.3 Development Methods & Contingencies 7
2. System design model 8
2.1 Subsystem decomposition 9
2.2 Hardware/software mapping 12
2.3 Access Control 14
3. OBJECT MODEL 17
3.1 Class Diagram – Refined Object Model 17
Purpose of the Class Diagram 17
Core Classes and Responsibilities 18
3.2 Sequence Diagram 20
ML-Assisted Request Classification - Sequence Diagram 1 20
Human-in-the-Loop Alert Review - Sequence Diagram 2 21
Offline Model Training and Evaluation – Sequence Diagram 3 22
Model Deployment and Rollback – Sequence Diagram 4 23
3.3 State Chart Diagram 25
4. Detailed Design 26
References 38

List of Tables

Table	Page
Table 4.1 ModSecurityAuditLogParser class	26
Table 4.2 Attributes description for ModSecurityAuditLogParser class	27
Table 4.3 Operation description for ModSecurityAuditLogParser class	27
Table 4.4 FeatureExtractor class	28
Table 4.5 Attributes description for FeatureExtractor class	28
Table 4.6 Operation description for FeatureExtractor class	29
Table 4.7 MLClassifier class	30
Table 4.8 Attributes description for MLClassifier class	30
Table 4.9 Operation description for MLClassifier class	31
Table 4.10 AlertPrioritizer class	31
Table 4.11 Attributes description for AlertPrioritizer class	32
Table 4.12 Operation description for AlertPrioritizer class	32
Table 4.13 ExplanationGenerator class	33
Table 4.14 Attributes description for ExplanationGenerator class	33
Table 4.15 Operation description for ExplanationGenerator class	34
Table 4.16 ModelTrainer class	34
Table 4.17 Attributes description for ModelTrainer class	35
Table 4.18 Operation description for ModelTrainer class	35
Table 4.19 Dashboard class	36
Table 4.20 Attribute Description for Dashboard class	36
Table 4.21 Operation Description for Dashboard class	37

List of figures

Figure	Page
UML Component Diagram (Subsystem Decomposition)	11
UML Deployment Diagram	13
Class Diagram	19
ML-Assisted Request Classification - Sequence Diagram 1	20
Human-in-the-Loop Alert Review - Sequence Diagram 2	21
Offline Model Training and Evaluation – Sequence Diagram 3	22
Model Deployment and Rollback – Sequence Diagram 4	23
General Sequence Diagram	24
State Chart Diagram	25

Definitions, Acronyms, Abbreviations

Abbreviation	Stands for
CRS	OWASP Core Rule Set
ML	Machine Learning
FP	False Positive
TP	True Positive
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
IPC	Inter-Process Communication

1. Introduction

1.1 Purpose

The purpose of this System Design document is to translate the functional requirements defined in the SRS into a concrete technical architecture for Modintel. It serves as the authoritative blueprint for developers, defining the internal structure of the system's core classes and their interactions. In particular, this document describes how the system bridges the gap between machine learning analysis and rule-based blocking. It ensures that the technical design supports the need for security analysts to verify automated predictions prior to their impact on system policy by directing the implementation of the "Human-in-the-Loop" workflow.

1.2 General Overview

System Context

The AI-Enhanced ModSecurity system (Modintel) operates as a modular extension to the standard ModSecurity Web Application Firewall. Rather than replacing the existing infrastructure, it functions as a decision-support layer. It intercepts audit logs generated by the OWASP Core Rule Set (CRS) and processes them through a secondary analysis pipeline to identify false positives.

Architecture and Design Approach

The software architecture follows a Hybrid Service-Oriented design. The system is divided into three distinct, loosely coupled subsystems:

1. **The Interception & Parsing Layer**: Utilizes the ModSecurityAuditLogParser to ingest raw logs and the Feature Extractor to convert complex HTTP metadata into numerical vectors suitable for analysis.
2. **The Intelligence Layer (ML Service)**: An independent Python-based service containing the ML Classifier. It performs inference using pre-trained models and assigns confidence scores to alerts.
3. **The Explainability & Prioritization Layer**: A critical architectural addition that includes the Alert Prioritizer to rank alerts for analysts and the Explanation Generator to provide human-readable justifications for every ML decision.

4. **The Presentation Layer:** A web-based dashboard that allows security analysts to visualize the data and provide feedback, closing the loop for future model training.

Design Goals

The technical design is driven by three main goals derived from the requirements:

- **Modularity:** By separating Log Parsing from Inference (as seen in the Class Diagram), we ensure that changes to the log format do not require rewriting the machine learning logic.
- **Traceability:** The design enforces strict audit trails. Every MLClassificationResult is inextricably linked to a ModSecurityAlert, ensuring that all automated advice can be audited.
- **Fail-Safe Stability**: The components are deployed such that a failure in the Python ML Service Node does not crash the Web Server Node, ensuring continuous application availability.

1.3 Development Methods & Contingencies

Development Methods:

The system design utilizes a mixed-methodology approach that fits the different technologies involved:

- **Object-Oriented Design (OOD):** The core logic is designed using OOD principles. Key functional areas are encapsulated into specific classes (e.g., ModelTrainer, MLClassifier) with defined attributes or and operations, promoting code reuse and easier testing.

- **UML Modeling:** The design relies heavily on Unified Modeling Language to define system behavior:
    - Sequence Diagrams are used to map the asynchronous flow between the Web Client, ModSecurity Engine, and the ML Inference Module.
    - State Chart Diagrams are used to strictly define the lifecycle of a security alert from "Generated" to "Resolved."

- **Structured Scripting:** The interface with ModSecurity is handled via structured Lua scripts, ensuring low-overhead communication with the C++ host process.ss, allowing us to refine the machine learning model repeatedly as we gather more test data.

Contingencies and Alternative Plans

- **Latency Constraints:** The design currently uses a Local Loopback (HTTP/REST) interface for communication between ModSecurity and the ML Service to ensure compatibility and ease of debugging.
    - Contingency: If performance testing reveals latency exceeding the 20ms threshold, the architecture allows the FeatureExtractor and MLClassifier to be refactored to use Unix Domain Sockets without altering the overarching object model.

- **Explainability Limitations:** There is a risk that the ExplanationGenerator may produce generic reasons if the underlying model is too complex.
    - Contingency: The design includes a fallback mechanism to prioritize "Rule-Based" explanations (citing the specific CRS rule description) if the "Feature-Based" explanation is statistically insignificant.

2. System design model

This section explains the overall architecture of Modintel. It shows how the system is divided into major parts (subsystems), what each part does, and how the software components are deployed on hardware.

The system is designed as an extension to ModSecurity, not a replacement. ModSecurity and the OWASP Core Rule Set (CRS) continue to handle all rule-based security decisions, while the machine learning components provide additional support to help reduce false positive alerts and improve alert prioritization.

2.1 Subsystem decomposition

The system is divided into several subsystems, each with a clear responsibility. This modular design makes the system easier to understand, maintain, and extend. It also ensures that if one component fails (such as the ML module), the core ModSecurity functionality is not affected.

2.1.1 Subsystem Descriptions

1. ModSecurity and OWASP CRS

- This is the core security subsystem.
- It inspects incoming HTTP requests using predefined CRS rules.
- When a rule is triggered, it generates alerts, anomaly scores, and audit logs.
- This subsystem always remains in control of enforcement decisions.
- The system is considered a security-critical application, requiring conservative failure handling and predictable behavior

2. Integration Layer

- Connects ModSecurity with the machine learning components.
- Extracts relevant data from ModSecurity audit logs.
- Sends this data to the ML inference module.
- Ensures that ModSecurity's original behavior is not changed.

3. Feature Extraction Module

- Converts ModSecurity audit logs into structured feature vectors.
- Extracted features include:
  - Triggered rule IDs
  - Anomaly scores
  - HTTP request metadata
- Uses the same feature format for both training and live inference to ensure consistency.

4. Machine Learning Inference Module

- Runs trained machine learning models.
- Evaluates flagged requests and produces:
  - A classification result
  - A confidence score indicating the likelihood of a false positive
- Operates only as a decision-support component, not an automatic blocker.

5. Alert Prioritization and Policy Engine

- Uses ML confidence scores together with administrator-defined thresholds.
- Helps rank and prioritize alerts for human review.
- Does not override ModSecurity rules or decisions.

6. Explanation Engine

- Generates human-readable explanations for ML-assisted decisions.
- Explains which rules and features influenced the classification.
- Helps analysts understand and trust the ML output.

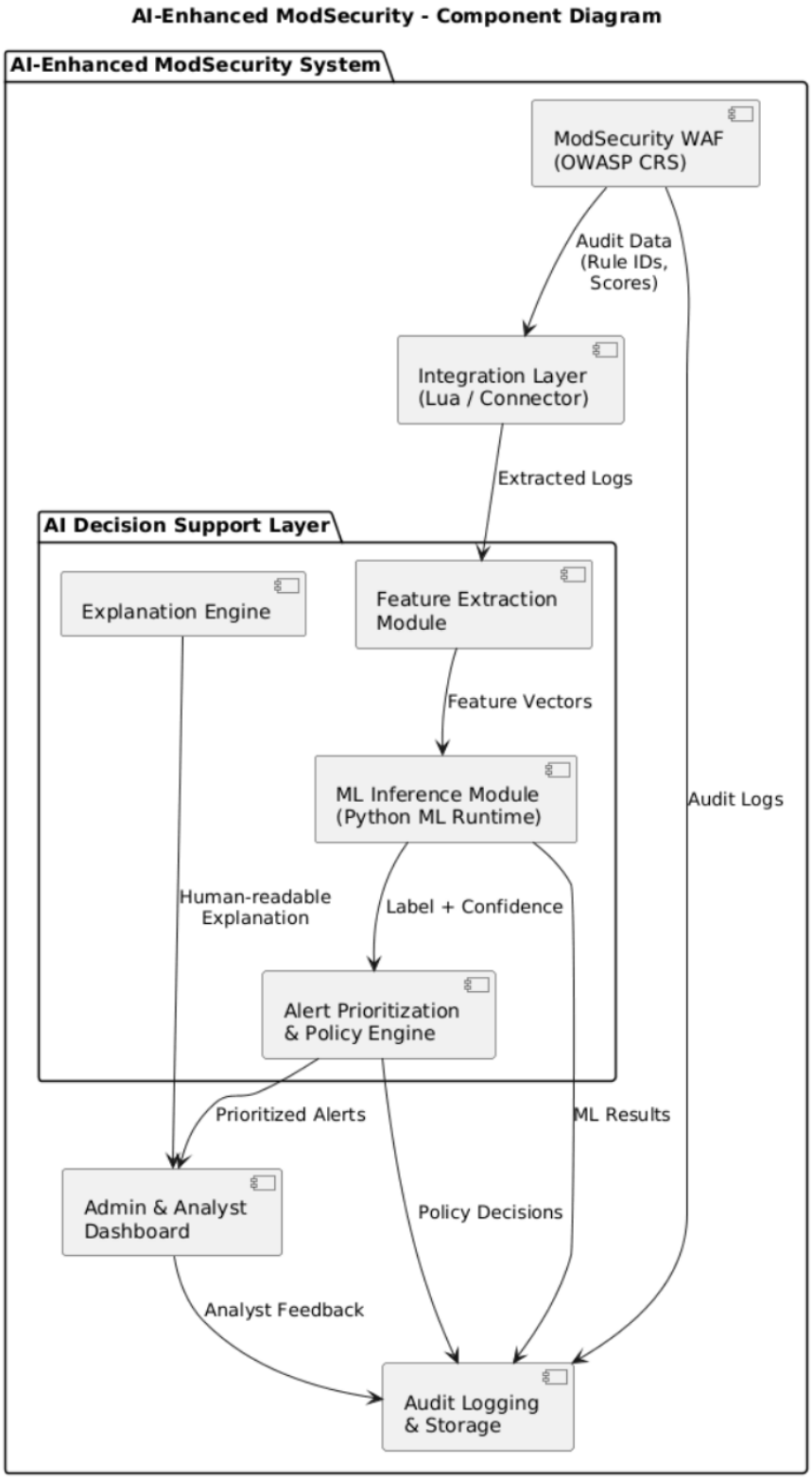
7. Audit Logging and Storage

- Stores all security-related records, including:
  - ModSecurity alerts
  - ML predictions
  - Analyst feedback
  - Policy changes
- Ensures full traceability for auditing and evaluation purposes.

8. Admin and Analyst Interface

- Provides a web-based dashboard for users.
- Allows security analysts to:
  - View alerts and confidence scores
  - Label alerts as true or false positives

2.1.2 UML Component Diagram (Subsystem Decomposition)



2.2 Hardware/software mapping

This section explains how the system's software components are deployed across hardware resources. The system is designed to run on standard server infrastructure without requiring specialized hardware.

2.2.1 Deployment Explanation

Web Server Node

- Hosts the protected web application.
- Runs ModSecurity with the OWASP CRS.
- Executes the integration logic that forwards data to the ML service.

ML Service Node

- l Runs the Python-based feature extraction, inference, and explanation modules.
- l Can run on the same machine or a separate server.

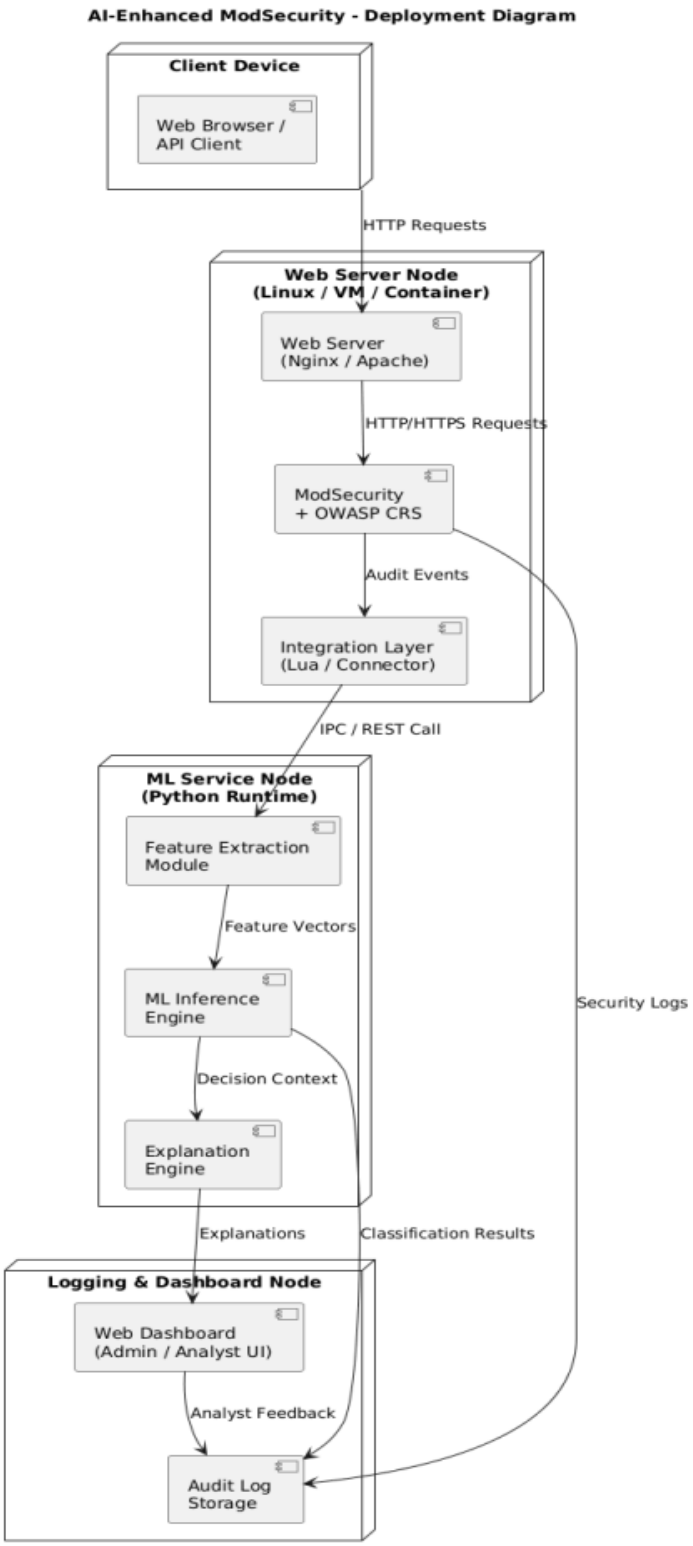
Logging and Dashboard Node

- l Stores audit logs and analyst feedback.
- l Hosts the web dashboard used by administrators and analysts.

Communication

- l Uses local IPC mechanisms or REST APIs.
- l Designed to keep latency low and avoid affecting request performance.

2.2.2 UML Deployment Diagram



2.3 Access Control

This section describes how access to Modintel is controlled. Because the system processes security alerts, machine learning results, and analyst feedback, access must be restricted to authorized users to ensure system integrity, traceability, and correct operation.

The system applies a role-based access control (RBAC) approach, where permissions are assigned according to user responsibilities. This model supports the system's human-in-the-loop design and ensures that machine learning components assist security decisions without independently enforcing them.

2.3.1 User Roles

Access to system functions is governed by the following user roles.

1. Security Analyst

Security Analysts are responsible for reviewing security alerts and machine-learning-assisted results.

Permitted Actions:

- View ModSecurity alerts and request metadata
- View machine learning classification results and confidence scores
- Access human-readable decision explanations
- Label alerts as:
    - True Positive
    - False Positive
    - Requires further investigation
- Review historical alerts and comparisons

Restrictions:

- Cannot modify ModSecurity rules
- Cannot deploy, retrain, or replace machine learning models

- Cannot change system policies or configurations

2. System Administrator

System Administrators are responsible for configuring and maintaining the system.

Permitted Actions:

- All Security Analyst permissions
- Configure alert prioritization thresholds and operational modes
- Enable or disable ML-assisted features
- Deploy and roll back validated machine learning models
- Manage system configuration parameters
- View system performance and monitoring information

Restrictions:

- Cannot bypass ModSecurity rule enforcement
- Cannot enable automatic blocking decisions based solely on ML output

3. System Developer / Maintainer

System Developers and Maintainers are responsible for maintaining and extending the system.

Permitted Actions:

- Access system logs for debugging and maintenance
- Modify system components in development or testing environments
- Maintain integration, feature extraction, and ML modules

Restrictions:

- No authority over live enforcement decisions
- Cannot modify operational policies in production
- Cannot override analyst or administrator actions

2.3.2 Authentication

- Access to the administrative and analyst interfaces requires authenticated access.
- Authentication is handled by the deployment environment or application-level mechanisms.
- The system is intended to operate in a controlled environment.

2.3.3 Authorization and Permission Enforcement

- User actions are checked against their assigned role.
- Only authorized roles may perform sensitive operations such as:
  - Alert labeling
  - Policy configuration
  - Model deployment
- Unauthorized actions are rejected and recorded in system logs.
- Authorization failures do not affect request inspection or enforcement.

2.3.4 Access Control for Internal Components

- Machine Learning Inference Module
  - Accessible only through the system's integration layer
  - Not directly accessible to users or external systems
- Audit Logs and Feedback Data
  - Access is restricted to authorized components and users
  - Feedback data is protected from unauthorized modification
- l Model Artifacts
  - Stored in protected locations
  - Modifiable only during controlled deployment operations

2.3.5 Auditing and Accountability

- All access-related actions are logged, including:
  - Alert labeling
  - Configuration changes
  - Model deployment and rollback events
- Audit records support traceability and review.
- Logging failures do not impact security enforcement or request processing.

2.3.6 Access Control Principles

The access control design follows these principles:

- Least Privilege: Users are granted only the permissions necessary to perform their tasks.
- Human-in-the-Loop: Machine learning assists decision-making without replacing human judgment.
- Fail-Safe Operation: If access control mechanisms fail, the system continues operating using standard rule-based enforcement.

3. OBJECT MODEL

3.1 Class Diagram – Refined Object Model

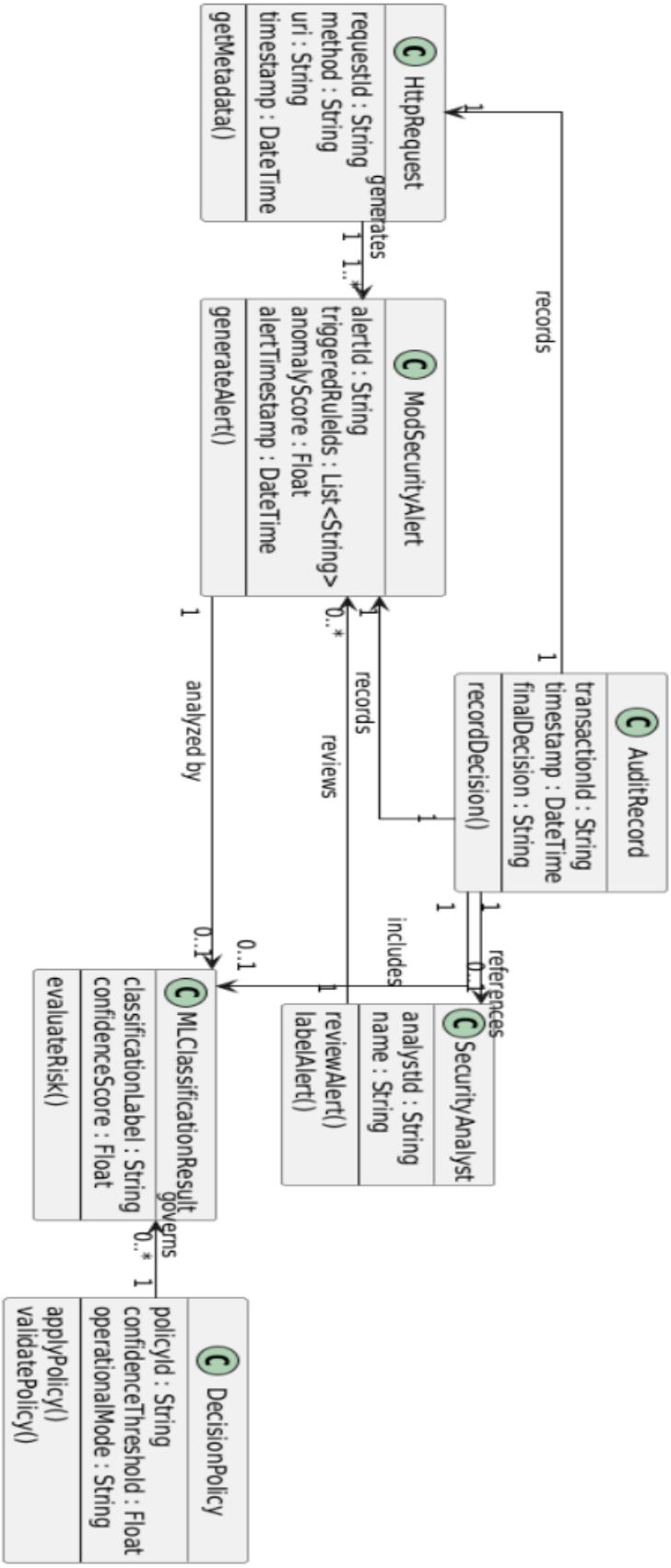
Purpose of the Class Diagram

Modintel's static structure is described by this class diagram model, which emphasizes the relationships between HTTP requests, rule-based alerts, machine learning classifications, human analyst actions, administrative policies, and audit records. It reflects the human-in-the-loop security design of the system and represents domain entities rather than implementation components.

Core Classes and Responsibilities

The main domain classes and their primary responsibilities are:

- HttpRequest: Captures incoming request details (requestId: String, method: String, uri: String, timestamp: DateTime). Provides metadata access via getMetadata().
- ModSecurityAlert: Represents rule-based detections (alertId: String, triggeredRules: List<String>, anomalyScore: Float, alertTimestamp: DateTime). Responsible for generating alerts via generateAlert().
- MLClassificationResult: Encapsulates machine learning advisory output (classificationLabel: String, confidenceScore: Float). Performs risk evaluation via evaluateRisk(). Modeled as advisory only — no direct enforcement authority.
- SecurityAnalyst: Represents the human analyst (analystId: String, name: String). Performs reviewAlert() and labelAlert() operations to enforce human-in-the-loop requirements.
- DecisionPolicy: Defines operational rules and thresholds (policyId: String, confidenceThreshold: Float, operationalMode: String). Supports applyPolicy() and validatePolicy().
- AuditRecord: Immutable record of the full decision chain (transactionId: String, timestamp: DateTime, finalDecision: String). Created via recordDecision() for traceability.

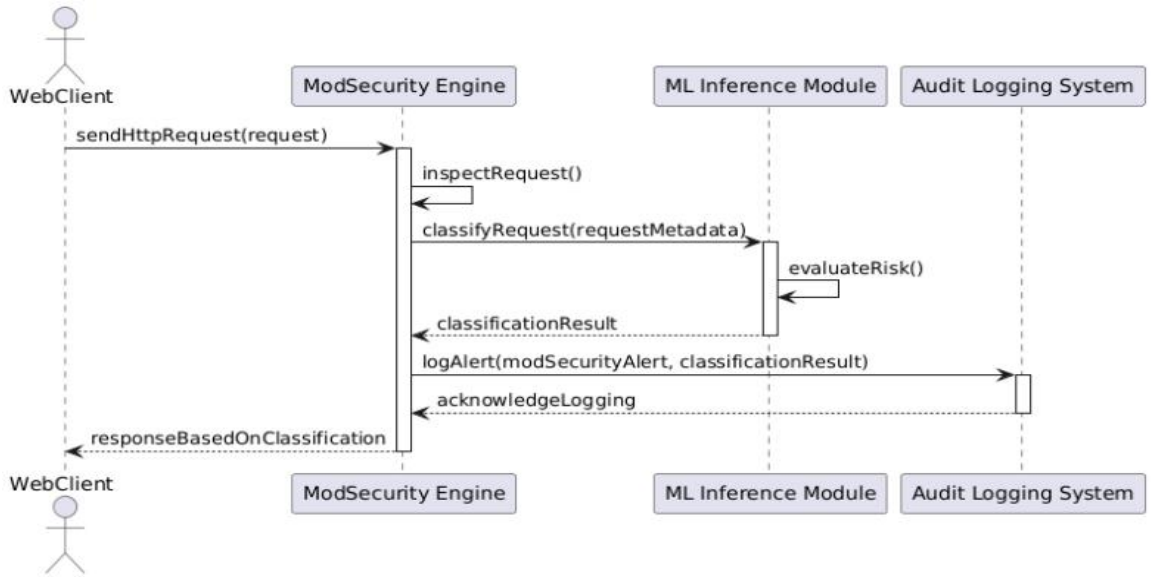

AuditRecord
transactionId : String
timestamp : DateTime
finalDecision : String
recordDecision()
records
1
1
1
references
0..1
SecurityAnalyst
analystId : String
name : String
reviewAlert()
labelAlert()
includes
1
1
HttpRequest
requestId : String
method : String
uri : String
timestamp : DateTime
getMetadata()
generates
1
1..*
ModSecurityAlert
alertId : String
triggeredRuleIds : List<String>
anomalyScore : Float
alertTimestamp : DateTime
generateAlert()
records
reviews
1
0..*
1
analyzed by
0..1
0..1
MLClassificationResult
classificationLabel : String
confidenceScore : Float
evaluateRisk()
governs
0..*
1
DecisionPolicy
policyId : String
confidenceThreshold : Float
operationalMode : String
applyPolicy()
validatePolicy()

Modeling Notes

- Machine learning outputs are modeled as advisory objects, not enforcement controllers.
- Human actions are explicitly represented to enforce human-in-the-loop constraints.
- Auditability is a first-class concern, modeled explicitly ratherthan implied.

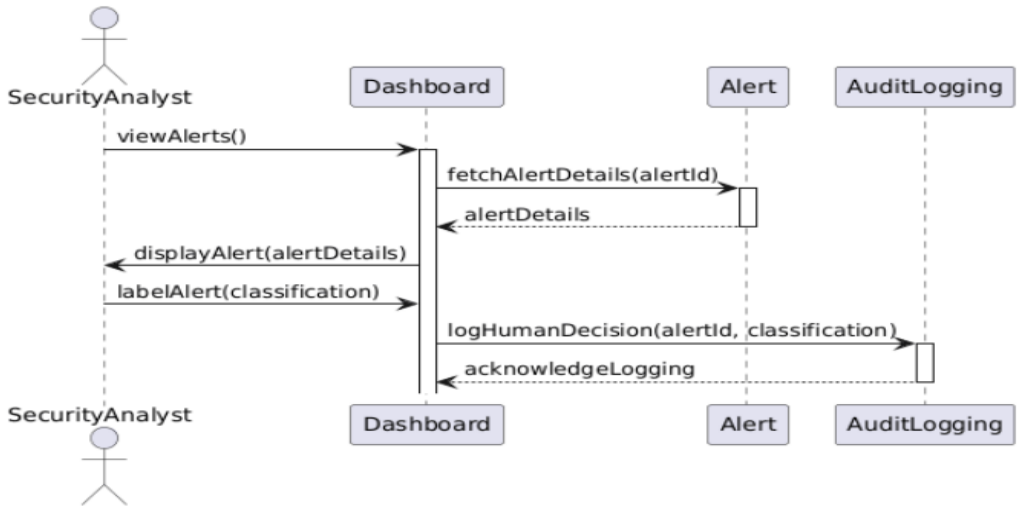
3.2 Sequence Diagram

ML-Assisted Request Classification - Sequence Diagram 1



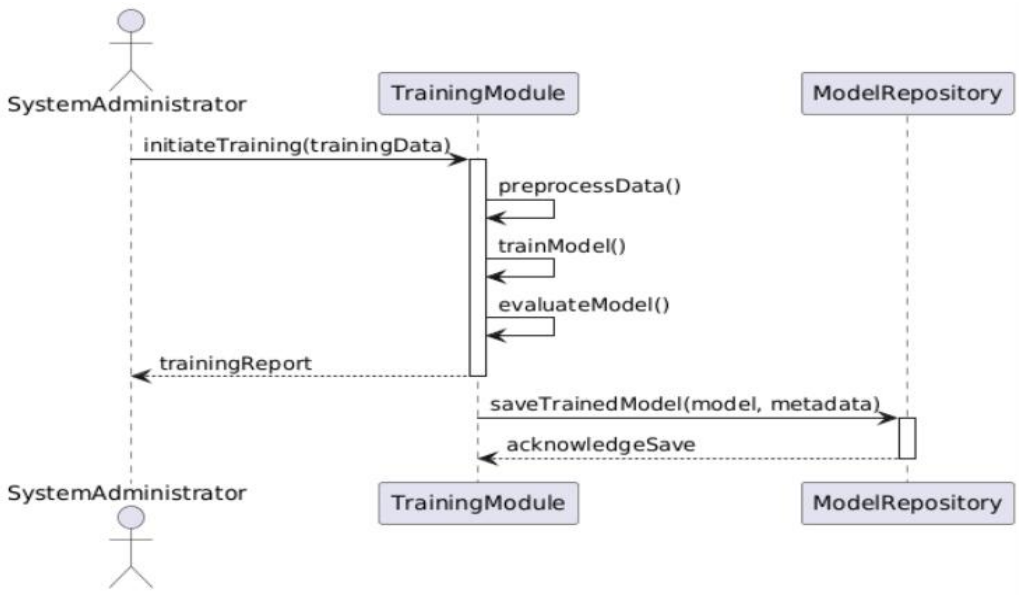
The enhanced ModSecurity system's complete request inspection and classification workflow is shown in this sequence diagram. When the WebClient submits an HTTP request to the ModSecurity Engine, the process starts. Using its rule set, the engine examines the request and extracts pertinent request metadata before sending it to the ML Inference Module for classification. The ModSecurity Engine receives a classification result from the ML module after it has assessed the request's risk. The ModSecurity Engine creates a security alert based on this outcome and sends it, along with the classification result, to the Audit Logging System for long-term storage. The ModSecurity Engine responds to the WebClient after logging is acknowledged; the final response is based on the security policies that have been applied and the classification outcome.

Human-in-the-Loop Alert Review - Sequence Diagram 2



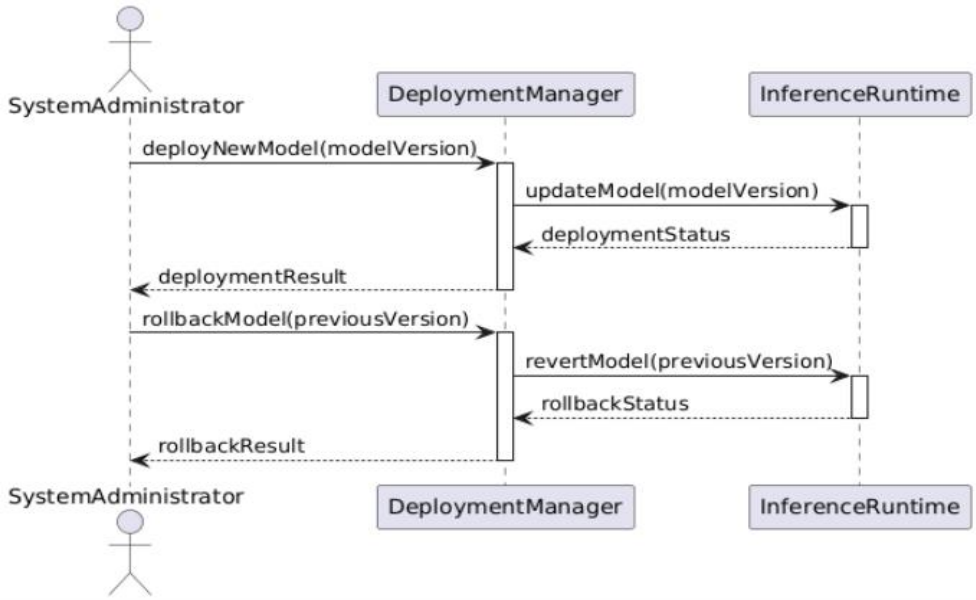
A security analyst's human-in-the-loop alert review procedure is depicted in the second sequence diagram. When the SecurityAnalyst accesses the dashboard to view generated security alerts, the interaction starts. The dashboard displays the alert details to the analyst after retrieving comprehensive information for a chosen alert from the Alert component. The SecurityAnalyst uses expert judgment to classify or label the alert after reviewing it. This human decision is then sent by the dashboard to the Audit Logging system, where the analyst's classification is continuously noted. This procedure guarantees that all analyst decisions are traceable for accountability, model feedback, and audit purposes, and that human expertise is added to automated detection results.

Offline Model Training and Evaluation – Sequence Diagram 3

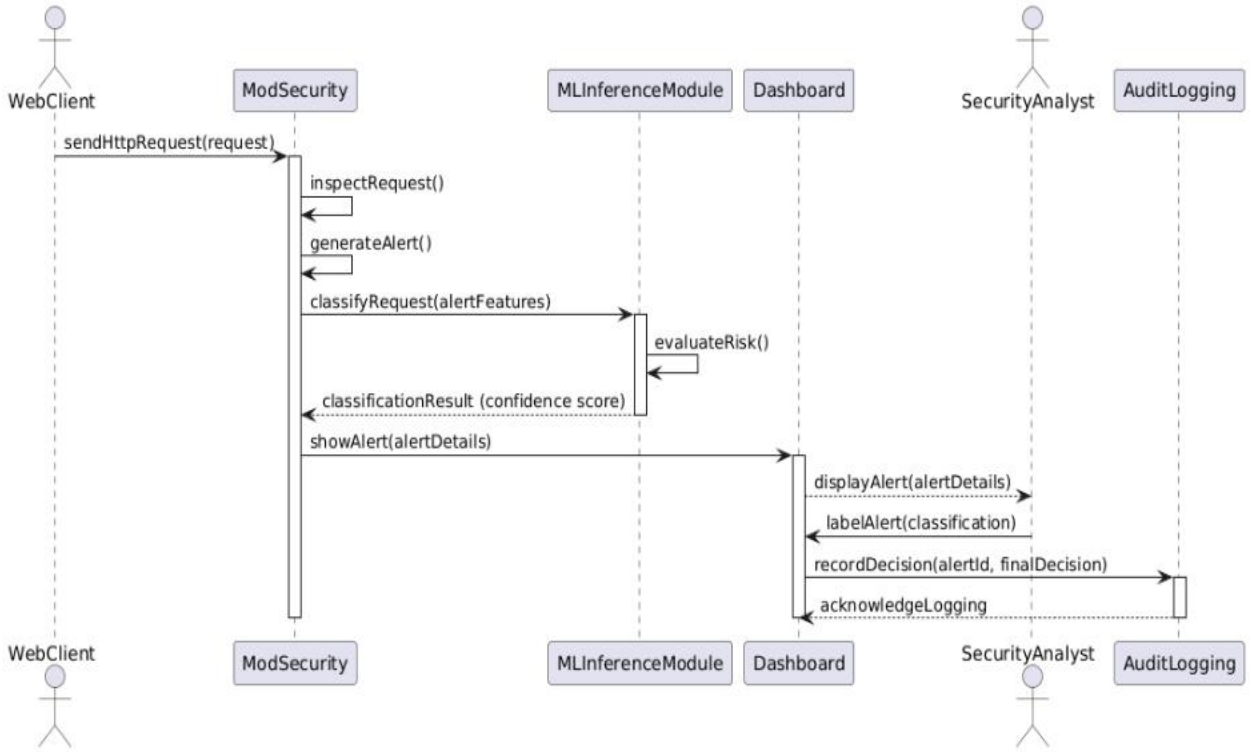


The offline machine learning model training and evaluation process overseen by a system administrator is depicted in this sequence diagram. The SystemAdministrator starts the training process by giving the TrainingModule training data, Following a successful evaluation, the trained model and its corresponding metadata are saved in the Model Repository for use in inference or deployment in the future. In order to ensure system stability and model version traceability, this sequence emphasizes that model training is an offline, controlled process that is separated from real-time request handling.

Model Deployment and Rollback – Sequence Diagram 4



The system's controlled deployment and rollback of models is shown in this sequence diagram. When the Administrator uses the DeploymentManager to request the deployment of a new model version, the process starts. To update the active model used for real-time inference, the deployment manager works in tandem with the Inference Runtime. The deployment status is returned by the runtime after the update is finished, and the administrator receives this information as the deployment outcome. The rollback scenario, in which the administrator submits a request to return to a model version that has already been deployed, is also depicted in the diagram. The deployment manager responds by giving the inference runtime instructions to restore the earlier model that was specified. The rollback status is returned and reported to the administrator, ensuring rapid recovery in case of performance degradation or operational issues. This sequence emphasizes safe model lifecycle management, version control, and operational resilience in the deployment pipeline.

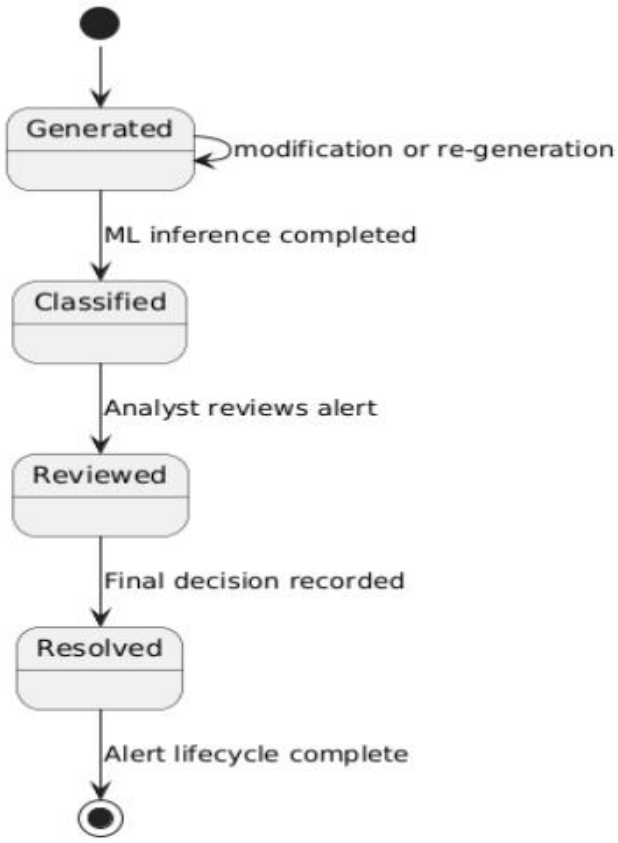


The entire system's operational flow, including modsecurity's initial operation, is shown in a single sequence diagram that incorporates automated request inspection, machine learning-based classification, human analysis, and audit logging. The ModSecurity component receives an HTTP request from a WebClient, examines it, and uses pre-established rules and anomaly detection logic to produce a security alert. The machine learning inference module receives pertinent alert features, assesses the risk, and provides a classification result and a confidence score.

Following automated classification, the alert and its associated details are forwarded to the dashboard, where they are presented to a security analyst. The analyst reviews the alert, applies expert judgment, and assigns a final classification. This final decision is then recorded in the audit logging system to ensure traceability, accountability, and support for future analysis or model improvement. The sequence highlights the system’s hybrid design, combining real-time automated defenses with human-in-the-loop validation and comprehensive auditing across the entire security workflow.

3.3 State Chart Diagram

The *ModSecurityAlert* object follows a clearly defined lifecycle, which can be modeled using a state chart diagram. An alert is initially Generated when ModSecurity detects a potential threat. Once the system performs ML inference, the alert transitions to the Classified state, where a confidence score is assigned. The alert is then Reviewed by a security analyst, who examines the alert details and provides a label or decision. Finally, the alert reaches the Resolved state after the analyst's decision is recorded in the audit log, completing its lifecycle.



4. Detailed Design

The detailed design section describes the internal structure of the primary classes that realize the core functionality of Modintel. The classes selected for detailed description are those directly responsible for the main data flow: log parsing, feature extraction, machine learning classification, alert prioritization, model explanation, and offline model training. These classes correspond to the central elements identified in the class diagram (Section 3.1) and the key functional requirements (Section 3.2 of the SRS). Supporting classes (such as DecisionPolicyManager, AuditLogger, and DashboardController) are shown in the class diagram but are not described in full detail here, as their behavior follows standard patterns and is less critical to the novel aspects of the system.

4.1 ModSecurityAuditLogParser class

Table: 4.1 ModSecurityAuditLogParser class

ModSecurityAuditLogParser
+logSource:String +format:String -currentEntry:Dictionary
+parseNextEntry() +getTriggeredRules() +getAnomalyScore() +extractRequestMetadata()

Table: 4.2 Attributes description for ModSecurityAuditLogParser class

Attribute	Type	Visibility	Invariant
logsource	String	Public	logSource ≠ NULL and must be a valid file path or stream identifier
format	String	Public	format ≠ NULL and must be one of "JSON" or "Native
currentEntry	Dictionary	Private	currentEntry ≠ NULL after successful parse; contains parsed log fields

Table: 4.3 Operation description for ModSecurityAuditLogParser class

Operation	Visibility	Return Type	Argument	Pre-Condition	Post-Condition
parseNextEntry	Public	Boolean	–	Log source is open and readable	Next entry parsed, or returns false if end of log is reached
getTriggeredRules	Public	List	–	Current entry is parsed	Returns list of fired CRS rule IDs
getAnomalyScore	Public	Float	–	Current entry is parsed	Returns total anomaly score from matched rules
extractRequestMetadata	Public	Dictionary	–	Current entry is parsed	Returns HTTP metadata (method, URI, IP, timestamp)

4.2 FeatureExtractor class

Table: 4.4 FeatureExtractor class

FeatureExtractor
+ruleEncoder:Dictionary +maxFeatures:Integer
+extract() +encodeRules() +normalizeScore()

Table: 4.5 Attributes description for FeatureExtractor class

Attribute	Type	Visibility	Invariant
ruleEncoder	Dictionary	Public	ruleEncoder ≠ NULL and contains mappings for at least 100 CRS rules
maxFeatures	Integer	Public	maxFeatures > 0 and maxFeatures ≤ 500 (to prevent excessive dimensions)

Table: 4.6 Operation description for FeatureExtractor class

Operation	Visibility	Return Type	Argument	Pre-Condition	Post-Condition
extract	Public	Array/Vector	Dictionary	Parsed audit data is available	Returns numerical feature vector ready for ML model
encodeRules	Public	Dictionary	List	List of rule IDs is provided	Returns encoded / count-based representation of rules
normalizeScore	Public	Float	Float	Raw anomaly score is provided	Returns normalized score in [0,1] range

4.3 MLClassifier class

Table: 4.7 MLClassifier class

MLClassifier
+model:MLModel +threshold:Float
+load() +classify()

Table: 4.8 Attributes description for MLClassifier class

Attribute	Type	Visibility	Invariant
model	MLModel	Private	model ≠ NULL after successful load; must be scikit-learn compatible
threshold	Float	Public	threshold > 0.0 and threshold ≤ 1.0 (confidence cutoff for false-positive decision)

Table: 4.9 Operation description for MLClassifier class

Operation	Visibility	Return Type	Argument	Pre-Condition	Post-Condition
load	Public	Boolean	String	Model file path is valid	Model is loaded into memory, or returns false on failure
classify	Public	(Integer, Float)	Array	Feature vector is provided	Returns (label: 0 = benign, 1 = attack; confidence probability)

4.4 AlertPrioritizer class

Table: 4.10 AlertPrioritizer class

AlertPrioritizer
+feedbackStore:Dictionary
+prioritize() +recordFeedback()

Table: 4.11 Attributes description for AlertPrioritizer class

Attribute	Type	Visibility	Invariant
feedbackStore	Dictionary	Private	feedbackStore ≠ NULL; maps transaction ID to label ("FP" or "TP")

Table: 4.12 Operation description for AlertPrioritizer class

Operation	Visibility	Return Type	Argument	Pre-Condition	Post-Condition
prioritize	Public	List	List of alerts	ML confidence scores available	Returns sorted list (highest likely false positives first)
recordFeedback	Public	Boolean	String, String	Transaction ID and label provided	Label stored in feedbackStore, or returns false on error

4.5 ExplanationGenerator class

Table: 4.13 ExplanationGenerator class

ExplanationGenerator
+featureImportance:Dictionary +ruleMetadata:Dictionary
+generateExplanation() +formatRuleContribution()

Table: 4.14 Attributes description for ExplanationGenerator class

Attribute	Type	Visibility	Invariant
featureImportance	Dictionary	Private	featureImportance ≠ NULL after classification; maps feature index to importance score
ruleMetadata	Dictionary	Private	ruleMetadata ≠ NULL and contains at least rule ID, category, and description for top CRS rules

Table: 4.15 Operation description for ExplanationGenerator class

Operation	Visibility	Return Type	Argument	Pre-Condition	Post-Condition
generateExplanation	Public	String	Integer label, Float confidence, Array features, List rules	Classification result and features available	Returns human-readable explanation string (e.g., "High FP confidence due to rules 942100 and 981173...")
formatRuleContribution	Public	String	List rules, Dictionary importance	Rule list and importance scores available	Formats contribution of individual rules in natural language

4.6 ModelTrainer class

Table: 4.16 ModelTrainer class

ModelTrainer
+dataset:DataFrame +algorithms:List
+trainAndEvaluate() +saveBestModel() +computeMetrics()

Table: 4.17 Attributes description for ModelTrainer class

Attribute	Type	Visibility	Invariant
dataset	DataFrame	Private	dataset <> NULL and must contain at least 1000 labeled samples with features and labels
algorithms	List	Public	algorithms contains at least one valid algorithm name ("RandomForest", "SVM", "LogisticRegression", etc.)

Table: 4.18 Operation description for ModelTrainer class

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
trainAndEvaluate	Public	Dictionary	String algorithm, Float testSize	Labeled dataset is loaded	Returns model performance metrics (accuracy, precision, recall, FPR)
saveBestModel	Public	Boolean	String path	Best model identified from evaluation	Serializes best model to file or returns false on failure
computeMetrics	Public	Dictionary	Model model, Array X_test, Array y_test	Trained model and test data available	Computes and returns standard classification metrics dictionary

4.7 Attributes Description for Dashboard Class

Table: 4.19 Dashboard class

DashboardController
+ activeUser: User + alertList: List +selectedAlertID: String
+renderAlerts() +displayConfidenceScores() +requestExplanation()

Table: 4.20 Attribute Description for Dashboard class

Attribute	Type	Visibility	Invariant
activeUser	User	Public	Must be authenticated
alertList	List	Private	Must contain valid alert objects
selectedAlertID	String	Private	Must reference an existing alert

Table: 4.21 Operation Description for Dashboard class

Operation	Visibility	Return Type	Argument	Pre-Condition	Post-Condition
renderAlerts	Public	void	–	Alerts exist	Alerts displayed
displayConfidenceScores	Public	void	–	ML results exist	Scores displayed
requestExplanation	Public	String	alertID	Explanation exists	Explanation displayed

References

- Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley. (Classic, concise reference for UML modeling used in your class, sequence, and state chart diagrams.)
- Folini, C., & Ristić, I. (2017). ModSecurity Handbook, Second Edition. Feisty Duck. (The definitive guide to ModSecurity, covering installation, configuration, and rule writing.)
- OWASP Foundation. (2025) - OWASP ModSecurity Core Rule Set Project. Retrieved from https://github.com/coreruleset/coreruleset
- https://PlatUML.com web resource to build diagrams
- SRS documentation